BHV_Loiter

Loiter behavior.

Included MIT MOOS-IvP PDF sources:

- bhv_loiter.pdf

The Loiter Behavior

Fall 2024

Michael Benjamin, mikerb@mit.edu
Department of Mechanical Engineering
MIT, Cambridge MA 02139
`project-pavlab/bhvdocs/bhv_loiter`

Contents

1 The Loiter Behavior

A behavior for transiting to and repeatedly traversing a set of waypoints forming a convex polygon. A similar effect can be achieved with the Waypoint behavior but this behavior assumes a set of waypoints forming a convex polygon to exploit certain useful algorithms discussed below. It also utilizes the non-monotonic arrival criteria used in the Waypoint behavior to avoid loop-backs upon waypoint near-misses. It also robustly handles dynamic exit and re-entry modes when or if the vehicle diverges from the loiter region due to external events. t is dynamically reconfigurable to allow a mission control module to repeatedly reassign the vehicle to different loiter regions by using a single persistent instance of the behavior.

1.1 Configuration Parameters

Listing 1.1: Configuration Parameters Common to All Behaviors.

`activeflag`: A MOOS variable-value pair posted when the behavior is in the *active* state. [more].

`condition`: Specifies a condition that must be met for the behavior to be running. [more].

`duration`: Time in behavior will remain running before declaring completion. [more].

`duration_idle_decay`: When true, duration clock is running even when in the *idle* state. [more].

`duration_reset`: A variable-pair such as `MY_RESET=true`, that will trigger a duration reset. [more].

`duration_status`: The name of a MOOS variable to which the vehicle duration status is published. [more].

endflag:	A MOOS variable-value pair posted when the behavior has completed. [more].
idleflag:	A MOOS variable-value pair posted when the behavior is in the *idle* state. [more].
inactiveflag:	A MOOS variable-value posted when the behavior is *not* in the *active* state. [more].
name:	The (unique) name of the behavior. [more].
nostarve:	Allows a behavior to assert a maximum staleness for a MOOS variable. [more].
perpetual:	If true allows the behavior to to run even after it has completed. [more].
post_mapping:	Re-direct behavior output normally to one MOOS variable to another instead. [more].
priority:	The priority weight of the behavior. [more].
pwt:	Same as priority.
runflag:	A MOOS variable and a value posted when a behavior has met its conditions. [more].
spawnflag:	A MOOS variable and a value posted when a behavior is spawned. [more].
spawnxflag:	A MOOS variable and a value posted when a behavior is spawned. [more].
templating:	Turns a behavior into a template for spawning behaviors dynamically. [more].
updates:	A MOOS variable from which behavior parameter updates are read dynamically. [more].

Listing 1.2: Configuration Parameters for the Loiter Behavior.

Parameter	Description
acquire_dist:	Distance to polygon outside which the behavior will be in "acquire" mode. Section 1.3.7.
capture_radius:	The radius tolerance, in meters, for satisfying the arrival at a point. Section 1.3.9.
center_activate:	If true, center of polygon is set to present position when behavior activates. Section 1.3.2.
center_assign:	An x,y pair, in meters, indicating a (new) center of the loiter polygon. Section 1.3.2.
clockwise:	If true, loitering is done clockwise (the default setting). Section 1.3.11.
polygon:	Polygon about which the vehicle will traverse and loiter. Section 1.3.1.
post_suffix:	A string to add as a suffix to variables posted by this behaviors. Section 1.3.10.
radius:	An alias for capture_radius. Section 1.3.9.
slip_radius:	An "outer" capture radius. Arrival declared when the vehicle is in this range and the distance to the point begins to increase. Section 1.3.9.
speed:	Speed in meters per second. Section 1.3.4.

speed_alt:	An alternative desired speed (m/s) at which the vehicle travels through the points. Applies only when the use_alt_speed parameter is set to true. The default is -1, which indicates internally that it has not been set. Section 1.3.5.
(Introduced after release 15.5.)	
spiral_factor:	The degree of spiraling when activated at the center of the polygon. Section 1.3.6.
use_alt_speed:	If true then attempt to use the alternate speed set with the speed_alt parameter, if that speed is set to a non-negative value. The default is false. Section 1.3.5.
(Introduced after release 15.5.)	
visual_hints:	Hints for visual properties in variables posted intended for rendering. Section 1.3.12.
xcenter_assign:	A x-value, in meters, indicating a (new) x-position of the loiter polygon. Section 1.3.2.
ycenter_assign:	A y-value, in meters, indicating a (new) y-position of the loiter polygon. Section 1.3.2.

Listing 1.3: Example Configuration Block.

```
Behavior = BHV_Loiter
{
  // General Behavior Parameters
  // ---------------------------
  name         = transit               // example
  pwt          = 100                   // default
  condition    = MODE==LOITERING       // example
  updates      = LOITER_UPDATES        // example

  // Parameters specific to this behavior
  // ------------------------------------
       acquire_dist = 10               // default
     capture_radius = 3                // default
    center_activate = false            // default
          clockwise = true             // default
        slip_radius = 15               // default
              speed = 0                // default
      spiral_factor = -2               // default

            polygon = radial:: x=5,y=8,radius=20,pts=8  // example
        post_suffix = HENRY                             // example

      center_assign = 40,50            // example
     xcenter_assign = 40               // example
     ycenter_assign = 50               // example


       visual_hints = vertex_size  = 1             // default
       visual_hints = edge_size    = 1             // default
       visual_hints = vertex_color = dodger_blue   // default
       visual_hints = edge_color   = white         // default
       visual_hints = nextpt_color = yellow        // default
       visual_hints = nextpt_lcolor = aqua         // default
       visual_hints = nextpt_vertex_size = 5       // default
       visual_hints = label        = zone3         // example
}
```

1.2 Variables Published

The below MOOS variables will be published by the behavior during normal operation, in addition to any configured flags. A variable published by any behavior may be suppressed or changed to a different variable name using the post_mapping configuration parameter described in Section **??**.

- LOITER_ACQUIRE: Posts 1 when in the ”acquire” mode, 0 otherwise.
- LOITER_DIST_TO_POLY: Current distance, in meters, to the loiter polygon.
- LOITER_ETA_TO_POLY: Estimated time of arrival to the polygon at present speed and trajectory.
- LOITER_INDEX: The index of the vertex in the loiter polygon currently heading toward.
- LOITER_MODE: A string indicating details of the acquire mode, e.g., "acquiring_external".
- LOITER_REPORT: A status string with current mode, current vertex, and prior arrivals.
- VIEW_POINT: A visual cue for rendering the next point in the loiter polygon.
- VIEW_POLYGON: A visual cue for rendering the loiter polygon.

The following are some examples:

```
       LOITER_REPORT = index=5,capture_hits=51,nonmono_hits=0,acquire_mode=false
        LOITER_INDEX = 5
      LOITER_ACQUIRE = 1
LOITER_DIST_TO_POLY = 2.2
 LOITER_ETA_TO_POLY = 294.4
        LOITER_MODE = stable
       VIEW_POLYGON = edge_size,0.0:vertex_size,0.0:0,-50:0,-150:150,-150:150,-50
```

1.3 Detailed Discussions on Loiter Behavior Parameters

1.3.1 The `polygon` Parameter for Setting the Loiter Region

The Loiter behavior is configured with a *loiter region*, defined by a convex polygon. The behavior will influence the vehicle to repeatedly traverse the set of points on the polygon. The polygon is specified typically in the behavior configuration block. Any string that properly defines a convex polygon is acceptable. The following two configuration lines, for example, will result in the same polygon.

```
polygon = 20,-40:40,-75:20,-110:-20,-110:-40,-75:-20,-40:label,Lima

polygon = format=radial, x=0, y=-75, radius=40, pts=6, snap=1, label=Lima
```

Each would produce the polygon shown in Figure 1:

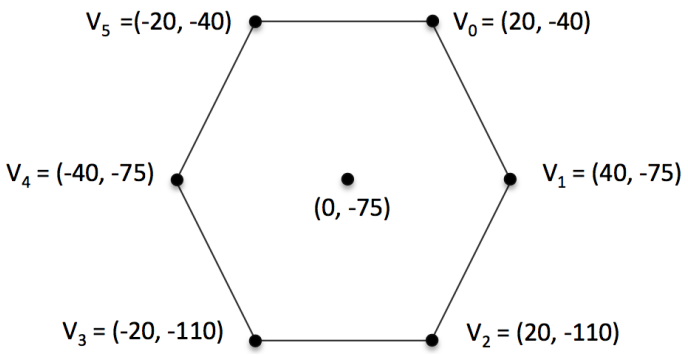

Figure 1: A typical loiter polygon with six vertices.

The shape and position polygon may be alterered dynamically (after the helm is launched and running) in one of two manners by either specifying new parameters explicitly, or by tying the loiter position to the vehicle position when the loiter behavior enters the running behavior mode.

1.3.2 The `updates` Parameter for Updating the Loiter Region

In the first case, altering the polygon parameter dynamically is accomplished by using the standard `updates` parameter described in Section **??**. For example, the behavior may be configured to receive

new updates by adding the following line to its configuration block:

```
updates = NEW_CONFIG
```

Its position may be then moved 75 meters North by posting the following to the MOOSDB:

```
NEW_CONFIG = "polygon = format=radial, x=0, y=0, radius=40, pts=6, snap=1, label=Lima"
```

Alternatively, if one wants to simply move the polygon to a new `(x,y)` position, the Loiter behavior also implements the `center_assign` configuration parameter. The same shift as above can be made without the source making the post having to know anything about the other parameters of the polygon:

```
NEW_CONFIG = "center_assign = 0,0"
```

Likewise, if one simply wants to move the polygon in either the x or y direction without knowing anything about the position in the other direction, the Loiter behavior implements the `xcenter_assign` and `ycenter_assign` parameters. Thus the above shift could also be accomplished without the source making the post knowing anything about the current x position of the polygon:

```
NEW_CONFIG = "ycenter_assign = 0"
```

1.3.3 The `center_activate` Parameter for Using the Current Vehicle Position

The Loiter behavior may also be configured to reset the (x,y) center position of the loiter polygon to the present position of the vehicle at the very the moment the behavior transitions from the idle to the running state. See Section **??** for more on behavior run states. This configuration is declared with the following line in the behavior configuration block:

```
center_activate = true
```

This setting is useful if one wants to, for example, send a command to the vehicle to exit some other mission mode and simply loiter at its present location until further notice. Of course this configuration as well, may also be dynamically toggled through a variable specified in the `updates` parameter.

1.3.4 The `speed` Parameter for Setting the Loiter Speed

The desired speed, in meters/second, at which the vehicle travels through the points.

1.3.5 The `speed_alt` and `use_alt_speed` Parameters

When the `use_alt_speed` parameter is set to `"true"`, the loiter behavior will use speed set in the `speed_alt` parameter, if it has been set. By default `use_alt_speed` is `"false"` and `speed_alt` is set to -1. This provides a convenient way for other behaviors or apps to periodically influence the loiter behavior to a different speed. The other behavior or app does not need to read, remember and reset the loiter behavior back to its original speed, but can instead just toggle `use_alt_speed` to `false` to return the loiter back to its original speed.

1.3.6 The spiral_factor Parameter

A value in the range $[0, 100]$ indicating the degree to which the vehicle will spiral out to the polygon if started on the inside. A setting of zero indicates it will move directly to the first polygon vertex. This parameter only relates to the situation of starting inside the polygon. The default value is 98.

1.3.7 The acquire_dist Parameter

The acquire_dist parameter is the distance in meters between the vehicle and the polygon that will trigger the vehicle to return to *acquire* mode. This notion applies to the case where the vehicle is both inside and outside the polygon. The re-acquire algorithms are different however. The default is 10 meters.

1.3.8 The xcenter_assign and ycenter_assign Parameters

1.3.9 The capture_radius and slip_radius Parameters

radius: The radius tolerance, in meters, for satisfying the arrival at a waypoint. As soon as the vehicle is within this distance to the waypoint the waypoint behavior begins operating on the next waypoint in the sequence, or completes and posts its endflags if there are no more waypoints.

slip_radius:As the vehicle progresses toward a waypoint, the sequence of measured distances to the waypoint decreases monotonically. The sequence becomes non-monotonic when it hits its waypoint or when there is a near-miss of the waypoint arrival radius. The slip_radius, a.k.a, the nm_radius, short for *non-monotonic radius* is an arrival radius distance within which a detection of increasing distances to the waypoint is interpreted as a waypoint arrival. This distance would have to be larger than the arrival radius to have any effect (see Figure **??**). As a rule of thumb, a distance of twice the arrival radius is practical.

1.3.10 The post_suffix Parameter

The post_suffix parameter names a string to be added as a suffix to each of the status variables posted by the behavior (LOITER_REPORT, LOITER_INDEX, LOITER_ACQUIRE, LOITER_DIST2POLY). By default, the suffix is the empty string and the variables will be posted as above. When multiple Loiter behaviors are configured in the helm it may help to distinguish the posted variabes by a suffix. A given suffix of "FOO" would result in the posting of LOITER_INDEX_FOO for example. The extra '_' character is inserted automatically.

1.3.11 The clockwise Parameter for Setting the Loiter Direction

The user may configure the direction the vehicle traverses the loiter polygon by setting the parameter:

```
clockwise = <value>
```

The possible case insensitive settings for <value> are "true", "false", "best". In the first two cases, the directions are explicitly set and will not vary regardless of the pose of the vehicle with respect to the polygon. In the case where clockwise=best, the direction is re-evaluated once, whenever the behavior run state transitions from idle to running. Thus the direction depends on the pose relative

position to the polygon at that particular point in time. This is shown in the lower case in Figure 2 below.

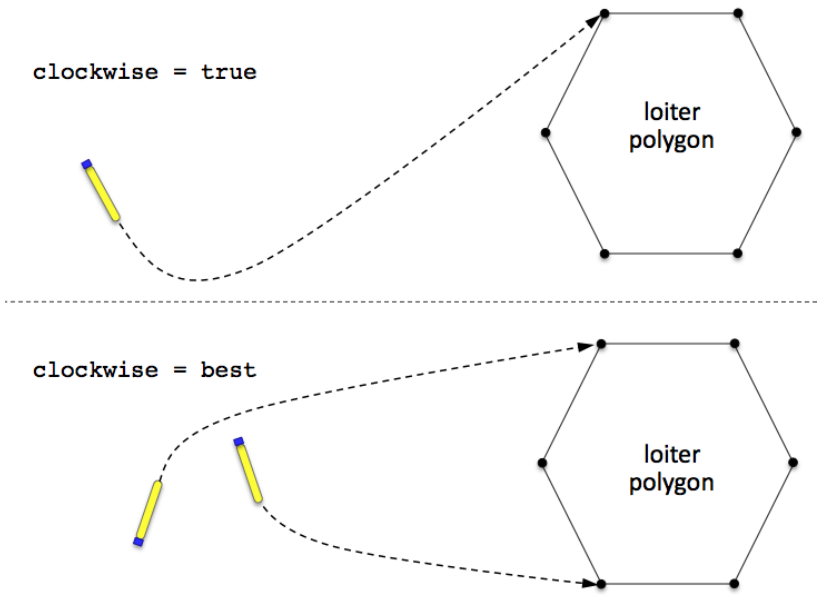

Figure 2: **The Loiter Direction.** The polygon traversal direction is determined by the `clockwise` configuration parameter. It may be explicitly set as shown on the top, or determined at run time when the behavior becomes non-idle as shown on the bottom.

Regardless of the prevailing direction, as the vehicle is transiting to the loiter polygon, the behavior will influence the vehicle toward the vertex that allows for the smoothest entry, given the chosen direction. If the polygon were a perfect circle, the vehicle would approach on one of the two tangent lines.

1.3.12 The `visual_hints` Parameter

Although the primary output of the Loiter behavior an IvP Function, a number of visual properties are also published for convenience in mission monitoring. This includes both the loiter polygon and the point on the polygon the behavior is presently progressing toward. These visual artifacts have default properties in size and color that may be altered to the user's preferences. These preferences are configurable through the `visual_hints` parameter. Each parameter below is used in the following way by example:

```
visual_hints = vertex_size=3, edge_size=2
visual_hints = vertex_color=khaki
```

- `vertex_size`: The size of vertices rendered in the loiter polygon. The default is 1.
- `edge_size`: The width of edges rendered in the loiter polygon. The default is 1.
- `vertex_color`: The color of vertices rendered in the loiter polygon. The default is "dodger_blue".

- edge_color: The color of edges rendered in the loiter polygon. The default is "white".
- nextpt_color: The color of the point rendered as the present next waypoint. The default is "yellow".
- nextpt_lcolor: The color of the label for the point rendered as the present next waypoint. The default is "aqua".
- label: The label of rendered for the loiter polygon.

Rendering of vertices or the next waypoint may be shut off with a size of zero, and labels may be shut off with the special color "invisible". For a list of legal colors, see Appendix **??**.

1.4 The Loiter Acquisition Mode

When the Loiter behavior is running (non-idle), it behaves differently depending on where the vehicle is in relation to the loiter polygon. The behavior has a notion of (a) when it is progressing in a "stable" manner around the polygon and (b) when it is trying to move the vehicle back to (a). The difference between the two is solely determined by whether or not the vehicle is within some *acquire distance* from the loiter polygon. This distance is set with the behavior configuration parameter:

```
acquire_dist = <distance>
```

The <distance> component must simply be a non-negative value. A value of zero should work fine, but essentially disables some useful ways in which the behavior may be coordinated with other parts of the mission. This relationship is shown in Figure 3.

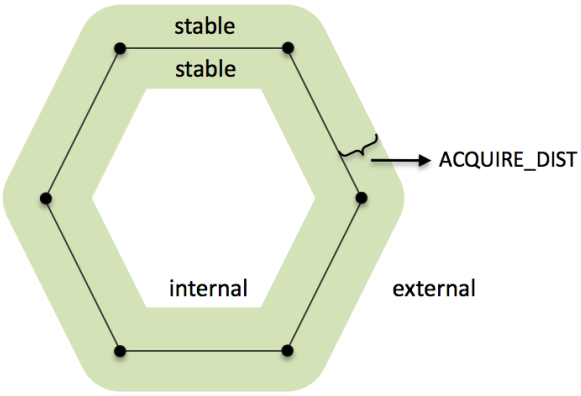

Figure 3: **The Loiter Polygon Zones:** The Loiter behavior regards itself to be in one of three distinct possible zones relative to the polygon boundary - *stable*, *internal*, *external*, depending on the setting of the acquire_dist parameter as shown.

When the vehicle is *not* within the stable zone, it is trying to acquire the polygon in one of four distint manners, depending on whether (a) the vehicle is internal or external and (b) whether it is "acquiring" the polygon for the first time, or whether it is "recovering" from having drifted out of the stable zone. The idea is depicted in Figure 4.

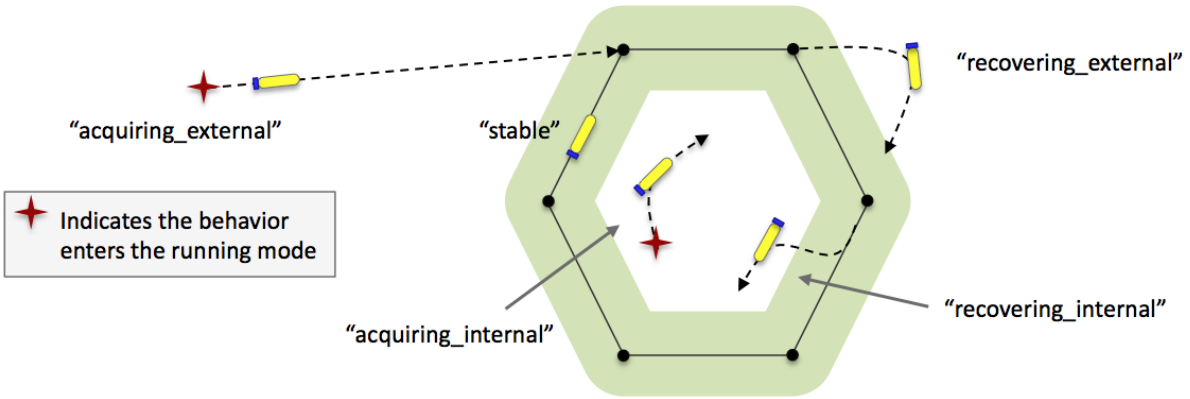

Figure 4: **Loiter Modes**: The behavior is one of five possible modes, *stable*, *acquiring_external*, *recovering_external*, *recovering_internal*, or *acquiring_internal*.

The current loiter mode is published by the behavior in the MOOS variable `LOITER_MODE`. As with any MOOS variable published by a behavior, this may be remapped to a different variable of the users liking with the `post_mapping` configuration parameter.

When the behavior is in any loiter mode other than the stable mode, it recalculates on each iteration which vertex it should be heading toward, the *approach vertex*. In the case of an external approach, the chosen vertex should remain steady unless there are external forces such as wind or current, or if the vehicle changes its aspect to the polygon significantly as it is executing a turn. In the case of an internal approach, the approach vertex will likely change during the approach, outward toward the polygon boundary, creating a pseudo outward spiral trajectory. Note that re-evaluating the approach vertex is not the same as re-evaluating the traversal direction of the polygon. The latter is only re-evaluated dynamically if the behavior is configured with `clockwise=best`, and then only when the behavior becomes non-idle.

The circumstance most common for triggering the acquire mode is the initial assignment to the vehicle to loiter at a new given region in the X,Y plane. This assignment *could* occur while the vehicle happens to already be within the polygon for a number of reasons. Furthermore, the vehicle could be driven off the polygon loiter trajectory due to environmental (wind or current) forces or the temporary dominance of other vehicle behaviors such as collision avoidance or tracking of another vehicle.

Once the behavior enters the acquire mode, it remains in this mode until arriving at the first waypoint (defined by the arrival and non-monotonic radii settings), after which it switches to normal mode until the acquire mode is re-triggered or the behavior run conditions are no longer met. There is currently no "complete" condition for this behavior other than a time-out which is defined for all behaviors.